COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S3
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/15/2026
NAME:	Payas, John Paul S.	**INSTRUCTOR:**	Sir. Jimlord Quejado.

ACTIVITY NO. 8 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

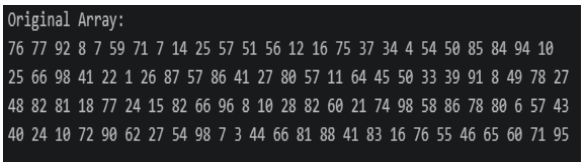
Code + Console Screenshot	
Observations	The program generated 100 random values. The values were initially unsorted and were used as the input for Shell Sort, Merge Sort, and Quick Sort.

Table 8-1. Array of Values for Sort Algorithm Testing

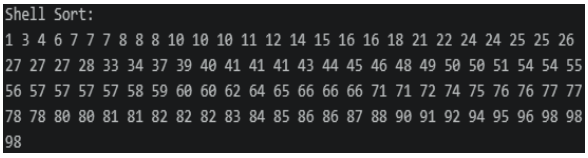
Code + Console Screenshot	
Observations	Shell Sort arranged the 100 random values in ascending order. It used different gaps to compare elements before performing the final sorting process.

Table 8-2. Shell Sort Technique

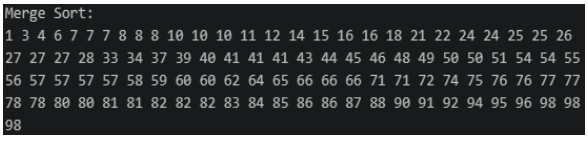
Code + Console Screenshot	
Observations	Merge Sort sorted the array in ascending order. It divided the array into smaller sections and then merged the sections back together in sorted order.

Table 8-3. Merge Sort Algorithm

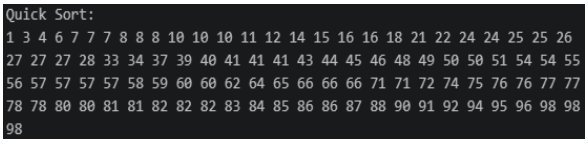
Code + Console Screenshot	
Observations	Quick Sort arranged the array in ascending order. It used a pivot to divide the array into smaller parts and recursively sorted those parts.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes because the left and right sublists of Quick Sort can be sorted using other sorting algorithms such as Insertion Sort, Selection Sort, or Merge Sort. For example, after partitioning, if the left sublist is {12, 5, 9} and the right sublist is {20, 17, 25}, Insertion Sort can be used to sort each sublist into {5, 9, 12} and {17, 20, 25}. This creates a hybrid sorting method where Quick Sort performs the partitioning while another sorting algorithm handles the smaller sublists.

Problem 2

Given:

{4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}

Answer:

Quick Sort can give fast time performance because it uses partitioning to divide the data into smaller sections and then sorts those sections recursively. Its average time complexity is **O(N log N)**, although its worst case can become **$O(N^2)$** when the pivot choices are poor.

Merge Sort has **O(N log N)** time complexity because the array is repeatedly divided into two halves, giving about log N levels, and the merging process at each level takes **O(N)** time. Quick Sort can also have **O(N log N)** average time complexity because balanced partitions create about log N levels, with each partition requiring approximately **O(N)** work.

Sorted Array

4, 12, 19, 25, 29, 30, 34, 43, 44, 48, 53, 67, 74, 87, 93

CONCLUSION

Summary of Lessons Learned:

In this activity, I learned how Shell Sort, Merge Sort, and Quick Sort work. I learned that Shell Sort improves insertion sort by using gaps, while Merge Sort uses divide and conquer to split and merge the data. I also learned that Quick Sort uses a pivot to partition the array into smaller sections.

Analysis of the Procedure:

The procedure showed how the same unsorted array can be processed using different sorting algorithms. Each algorithm successfully arranged the 100 values from smallest to largest. The activity also helped me understand how sorting algorithms use different methods to organize data efficiently.

Analysis of the Supplementary Activity:

The supplementary activity showed that Quick Sort can work together with other sorting algorithms for smaller sublists. I also learned why Merge Sort and Quick Sort can achieve O(N log N) performance, especially when the data is divided into balanced sections.

Concluding Statement / Feedback:

I was also able to compare their procedures and time complexity. My area for improvement is understanding the recursive process and choosing the most appropriate sorting algorithm for different types of data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It just fix my grammar but it was my idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.